MACHINE LEARNING

LAB RECORD

ITA0614 | EXPERIMENT 4

EXPERIMENT STATEMENT

Build an Artificial Neural Network by implementing the Backpropagation algorithm and test the same using appropriate data sets.

STUDENT NAME	REGISTER NUMBER
P H Devesh	**1911260236**
SUBJECT CODE	**SUBJECT NAME**
ITA0614	**Machine Learning**

Five program demonstrations • Executed Python examples • Verified output

SUB-PROGRAM 1

Backpropagation on XOR using a Neural Network

Problem Statement / Program Title

Backpropagation on XOR using a Neural Network.

Aim

Implement a small feed-forward artificial neural network and train it with backpropagation to learn the XOR mapping.

Algorithm

1. Define the XOR input-output pairs and initialize weights and biases with a fixed random seed.
2. Use sigmoid activation in the hidden and output layers.
3. Perform forward propagation to compute hidden and output activations.
4. Calculate output error and backpropagate the gradients through the output and hidden layers.
5. Update weights and biases using gradient descent for multiple epochs.
6. Print the initial and final mean squared error and the predicted output for each input.

Program

```
import numpy as np

np.random.seed(7)
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=float)
y = np.array([[0], [1], [1], [0]], dtype=float)

def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-z))

def sigmoid_derivative(a):
    return a * (1.0 - a)

input_size, hidden_size, output_size = 2, 4, 1
W1 = np.random.uniform(-1, 1, (input_size, hidden_size))
b1 = np.zeros((1, hidden_size))
W2 = np.random.uniform(-1, 1, (hidden_size, output_size))
b2 = np.zeros((1, output_size))
learning_rate = 0.8

def forward(inputs):
    hidden = sigmoid(inputs @ W1 + b1)
    output = sigmoid(hidden @ W2 + b2)
    return hidden, output

_, initial_pred = forward(X)
initial_loss = np.mean((y - initial_pred) ** 2)

for epoch in range(20000):
    hidden, output = forward(X)
    output_delta = (output - y) * sigmoid_derivative(output)
    hidden_delta = (output_delta @ W2.T) * sigmoid_derivative(hidden)
    W2 -= learning_rate * (hidden.T @ output_delta) / len(X)
    b2 -= learning_rate * output_delta.mean(axis=0, keepdims=True)
    W1 -= learning_rate * (X.T @ hidden_delta) / len(X)
    b1 -= learning_rate * hidden_delta.mean(axis=0, keepdims=True)

_, final_pred = forward(X)
final_loss = np.mean((y - final_pred) ** 2)
print("Training samples:", len(X))
print("Initial mean squared error:", round(float(initial_loss), 4))
print("Final mean squared error:", round(float(final_loss), 4))
print("Predictions (rounded):")
for inputs, pred in zip(X.astype(int), final_pred):
    print(f"{inputs.tolist()} -> {int(pred[0] >= 0.5)} (score={pred[0]:.3f})")
```

Input / Dataset

XOR truth table: inputs [0,0], [0,1], [1,0], [1,1] with targets 0, 1, 1, 0. A fixed seed initializes the network.

Output

```
Training samples: 4
Initial mean squared error: 0.2536
Final mean squared error: 0.0003
Predictions (rounded):
[0, 0] -> 0 (score=0.018)
[0, 1] -> 1 (score=0.981)
[1, 0] -> 1 (score=0.982)
[1, 1] -> 0 (score=0.018)
```

Result

Backpropagation reduced XOR mean squared error and learned the four expected XOR labels.

SUB-PROGRAM 2

Backpropagation for Binary Classification

Problem Statement / Program Title

Backpropagation for Binary Classification.

Aim

Train a neural network with backpropagation on a small binary dataset and report predictions and training accuracy.

Algorithm

1. Create a reproducible binary classification dataset with two numeric features.
2. Standardize the features so their scales are comparable.
3. Initialize a one-hidden-layer neural network's parameters.
4. Run forward propagation with sigmoid activation.
5. Compute gradients by backpropagation and update parameters with gradient descent.
6. Report final loss, accuracy on the training examples, and sample predictions.

Program

```
import numpy as np
from sklearn.preprocessing import StandardScaler

X = np.array([
    [1.0, 1.2], [1.3, 0.9], [0.8, 1.0], [1.1, 1.4],
    [3.0, 3.2], [2.7, 3.1], [3.3, 2.8], [3.1, 3.4]
])
y = np.array([[0], [0], [0], [0], [1], [1], [1], [1]], dtype=float)
scaler = StandardScaler()
X = scaler.fit_transform(X)

rng = np.random.default_rng(10)
W1 = rng.normal(0, 0.5, (2, 3)); b1 = np.zeros((1, 3))
W2 = rng.normal(0, 0.5, (3, 1)); b2 = np.zeros((1, 1))
lr = 0.3

def sigmoid(z):
    return 1 / (1 + np.exp(-np.clip(z, -30, 30)))

def forward(x):
    h = sigmoid(x @ W1 + b1)
    p = sigmoid(h @ W2 + b2)
    return h, p

for _ in range(5000):
    h, p = forward(X)
    d2 = (p - y) / len(X)
    d1 = (d2 @ W2.T) * h * (1 - h)
    W2 -= lr * (h.T @ d2); b2 -= lr * d2.sum(axis=0, keepdims=True)
    W1 -= lr * (X.T @ d1); b1 -= lr * d1.sum(axis=0, keepdims=True)

_, probabilities = forward(X)
predictions = (probabilities >= 0.5).astype(int)
accuracy = np.mean(predictions == y)
loss = -np.mean(y*np.log(probabilities+1e-9) + (1-y)*np.log(1-probabilities+1e-9))
print("Training examples:", len(X))
print("Final binary cross-entropy:", round(float(loss), 4))
print("Training accuracy:", f"{accuracy*100:.1f}%")
print("Actual labels:   ", y.astype(int).ravel().tolist())
print("Predicted labels:", predictions.ravel().tolist())
```

Input / Dataset

Eight synthetic two-feature records; first four belong to class 0 and last four belong to class 1. Features are standardized before training.

Output

```
Training examples: 8
Final binary cross-entropy: 0.001
Training accuracy: 100.0%
Actual labels:    [0, 0, 0, 0, 1, 1, 1, 1]
Predicted labels: [0, 0, 0, 0, 1, 1, 1, 1]
```

Result

The neural network correctly classified all eight synthetic training examples in this run; training accuracy is not a measure of generalization.

SUB-PROGRAM 3

Backpropagation for Multiclass Classification

Problem Statement / Program Title

Backpropagation for Multiclass Classification.

Aim

Implement a neural network with softmax output and backpropagation to classify three classes in a small synthetic dataset.

Algorithm

1. Generate a reproducible three-class dataset with numeric features.
2. Standardize the input features and one-hot encode the class labels.
3. Initialize weights for a hidden layer and a three-unit output layer.

4. Use sigmoid activation in the hidden layer and softmax at the output.

5. Compute cross-entropy gradients and update weights using backpropagation.

6. Report training accuracy and predictions for representative samples.

Program

```
import numpy as np
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler, LabelBinarizer
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

X, y = make_blobs(n_samples=90, centers=3, n_features=2,
                  random_state=12, cluster_std=1.0)
X = StandardScaler().fit_transform(X)
model = MLPClassifier(hidden_layer_sizes=(8,), activation="logistic",
                      solver="lbfgs", max_iter=2000, random_state=12)
model.fit(X, y)
pred = model.predict(X)
sample_predictions = model.predict(X[:6])
print("Dataset: synthetic three-class blobs")
print("Training examples:", len(X))
print("Classes:", sorted(set(y.tolist())))
print("Training accuracy:", f"{accuracy_score(y, pred)*100:.1f}%")
print("First 6 actual labels:   ", y[:6].tolist())
print("First 6 predicted labels:", sample_predictions.tolist())
```

Input / Dataset

Synthetic three-class blob dataset: 90 records, two numerical features, three classes; generated with fixed random_state=12.

Output

```
Dataset: synthetic three-class blobs
Training examples: 90
Classes: [0, 1, 2]
Training accuracy: 97.8%
First 6 actual labels:    [1, 0, 0, 0, 1, 0]
First 6 predicted labels: [1, 0, 0, 0, 1, 0]
```

Result

The MLP achieved 97.8% accuracy on the generated training data and correctly predicted the first six displayed examples.

SUB-PROGRAM 4

Backpropagation with a Validation Split

Problem Statement / Program Title

Backpropagation with a Validation Split.

Aim

Train a neural network with backpropagation and compare training and validation accuracy to illustrate generalization.

Algorithm

1. Create a reproducible binary dataset with informative features.

2. Split the data into training and validation subsets using stratification.

3. Standardize features using statistics fitted only on the training set.

4. Train an MLP classifier using backpropagation.

5. Evaluate both training and validation accuracy.

6. Report the sample counts and metrics, noting that scores depend on the generated dataset and split.

Program

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.pipeline import make_pipeline
from sklearn.metrics import accuracy_score

X, y = make_classification(
    n_samples=240, n_features=6, n_informative=4, n_redundant=1,
    n_classes=2, class_sep=1.2, random_state=21
)
X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.25, random_state=21, stratify=y
)
model = make_pipeline(
    StandardScaler(),
    MLPClassifier(hidden_layer_sizes=(8,), activation="logistic",
                  solver="adam", max_iter=1500, random_state=21,
                  early_stopping=False)
)
model.fit(X_train, y_train)
train_pred = model.predict(X_train)
val_pred = model.predict(X_val)
print("Dataset: synthetic binary classification")
print("Training samples:", len(X_train))
print("Validation samples:", len(X_val))
print("Training accuracy:", f"{accuracy_score(y_train, train_pred)*100:.1f}%")
print("Validation accuracy:", f"{accuracy_score(y_val, val_pred)*100:.1f}%")
print("First 8 validation predictions:", val_pred[:8].tolist())
```

Input / Dataset

Synthetic binary dataset of 240 records and 6 features. A stratified 75/25 split yields 180 training and 60 validation samples.

Output

```
Dataset: synthetic binary classification
Training samples: 180
Validation samples: 60
Training accuracy: 88.3%
Validation accuracy: 95.0%
First 8 validation predictions: [1, 1, 1, 1, 0, 0, 0, 1]
```

Result

The model achieved 88.3% training accuracy and 95.0% validation accuracy on the fixed synthetic split; the validation result is specific to this run and dataset.

SUB-PROGRAM 5

Backpropagation for Handwritten Digit Classification

Problem Statement / Program Title

Backpropagation for Handwritten Digit Classification.

Aim

Use a neural network trained by backpropagation to classify digit images from scikit-learn's built-in digits dataset.

Algorithm

1. Load the built-in handwritten digits dataset.
2. Scale pixel intensities to the range 0 to 1 and split into training and test sets.
3. Train an MLP classifier whose weights are optimized by backpropagation.
4. Predict the digit labels for the held-out test set.
5. Calculate test accuracy and display predictions for the first ten test images.
6. Treat the displayed score as a reproducible test-set result for this split.

Program

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

digits = load_digits()
X = digits.data / 16.0
y = digits.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)
model = MLPClassifier(
    hidden_layer_sizes=(32,), activation="relu", solver="adam",
    max_iter=500, random_state=42, early_stopping=True
)
model.fit(X_train, y_train)
pred = model.predict(X_test)
print("Dataset: scikit-learn handwritten digits")
print("Training samples:", len(X_train))
print("Test samples:", len(X_test))
print("Test accuracy:", f"{accuracy_score(y_test, pred)*100:.2f}%")
print("First 10 actual digits:   ", y_test[:10].tolist())
print("First 10 predicted digits:", pred[:10].tolist())
```

Input / Dataset

Built-in scikit-learn digits dataset: 1,797 images represented as 64 pixel features; 75% training and 25% held-out test split.

Output

```
Dataset: scikit-learn handwritten digits
Training samples: 1347
Test samples: 450
Test accuracy: 89.56%
First 10 actual digits:    [1, 0, 9, 1, 5, 6, 7, 9, 1, 5]
First 10 predicted digits: [8, 0, 9, 1, 5, 6, 7, 9, 1, 5]
```

Result

The MLP achieved 89.56% accuracy on the held-out digits test set. One of the first ten examples was misclassified, illustrating that test accuracy is not perfect.